

```

# a2a/system_integration.py
from datetime import datetime, timedelta
import asyncio
from typing import Dict, List, Optional

class A2ASystem:
    """Complete A2A system integration"""

    def __init__(self):
        self.skill_registry = skill_registry
        self.task_manager = task_manager
        self.agent_hierarchy = agent_hierarchy
        self.a2a_protocol = a2a_protocol
        self.agent_cards: Dict[str, AgentCard] = {}

        # Initialize core skills
        self._initialize_core_skills()

    def _initialize_core_skills(self):
        """Initialize common skills for the system"""

        # Code Generation Skill
        code_gen_skill = self.skill_registry.register_skill(
            name="Code Generation",
            description="Generate and optimize code based on requirements and specifications",
            keywords=["programming", "development", "code", "generation"],
            sample_prompt="Generate a Python function that calculates fibonacci sequence up to n
numbers",
            supported_media=[MediaType.TEXT_PLAIN, MediaType.APPLICATION_JSON],
            skill_type="coding",
            input_schema={
                "type": "object",
                "properties": {
                    "requirements": {"type": "string"},
                    "programming_language": {"type": "string"},
                    "code_style": {"type": "string"},
                    "optimization_level": {"type": "string", "enum": ["low", "medium", "high"]}
                },
                "required": ["requirements", "programming_language"]
            },
            output_schema={
                "type": "object",
                "properties": {
                    "generated_code": {"type": "string"},

```

```

        "explanation": {"type": "string"},
        "complexity_analysis": {"type": "string"}
    }
}
)

# Security Analysis Skill
security_skill = self.skill_registry.register_skill(
    name="Security Analysis",
    description="Analyze code and systems for security vulnerabilities and compliance
issues",
    keywords=["security", "vulnerability", "analysis", "compliance"],
    sample_prompt="Analyze this Python code for security vulnerabilities and suggest
fixes",
    supported_media=[MediaType.TEXT_PLAIN, MediaType.APPLICATION_JSON],
    skill_type="security",
    input_schema={
        "type": "object",
        "properties": {
            "code_to_analyze": {"type": "string"},
            "security_standards": {"type": "array", "items": {"type": "string"}},
            "risk_level": {"type": "string", "enum": ["low", "medium", "high", "critical"]}
        },
        "required": ["code_to_analyze"]
    },
    output_schema={
        "type": "object",
        "properties": {
            "vulnerabilities": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "severity": {"type": "string"},
                        "description": {"type": "string"},
                        "line_number": {"type": "integer"},
                        "suggestion": {"type": "string"}
                    }
                }
            },
            "overall_risk": {"type": "string"},
            "compliance_status": {"type": "string"}
        }
    }
}

```

```

    )

def register_agent(self, base_name: str, agent_type: str,
                  parent_agent_id: Optional[str] = None,
                  capabilities: List[str] = None) -> AgentIdentity:
    """Register a new agent with the system"""

    agent_identity = self.agent_hierarchy.register_agent(
        base_name, agent_type, parent_agent_id, capabilities
    )

    # Create agent card
    agent_card = AgentCard(
        agent_id=agent_identity.agent_id,
        parent_agent_id=agent_identity.parent_agent_id,
        agent_type=agent_identity.agent_type,
        security_clearance="LEVEL_2", # Default clearance
        capabilities=agent_identity.capabilities
    )

    self.agent_cards[agent_identity.agent_id] = agent_card

    return agent_identity

def add_skill_to_agent(self, agent_id: str, skill: Skill):
    """Add a skill to an agent's capabilities"""

    if agent_id in self.agent_cards:
        self.agent_cards[agent_id].skills[skill.skill_id] = skill

        # Update agent capabilities
        if skill.name not in self.agent_cards[agent_id].capabilities:
            self.agent_cards[agent_id].capabilities.append(skill.name)

def create_agent_task(self, task_name: str, description: str, sender_agent_id: str,
                    destination_agent_id: str, flow_direction: FlowDirection,
                    required_skill_types: List[str], input_data: Dict = None,
                    priority: int = 1) -> Optional[Task]:
    """Create a task that requires specific skills"""

    # Find skills matching required types
    required_skills = []
    for skill_type in required_skill_types:
        skills = self.skill_registry.find_skills_by_keyword(skill_type)

```

```

    if skills:
        required_skills.append(skills[0].skill_id) # Use first matching skill

    if not required_skills:
        print(f"No skills found for types: {required_skill_types}")
        return None

    # Verify destination agent has required skills
    destination_card = self.agent_cards.get(destination_agent_id)
    if not destination_card:
        print(f"Destination agent {destination_agent_id} not found")
        return None

    agent_skill_ids = set(destination_card.skills.keys())
    required_skill_set = set(required_skills)

    if not required_skill_set.issubset(agent_skill_ids):
        missing_skills = required_skill_set - agent_skill_ids
        print(f"Destination agent missing skills: {missing_skills}")
        return None

    # Create task
    task = self.task_manager.create_task(
        task_name=task_name,
        description=description,
        sender_agent_id=sender_agent_id,
        destination_agent_id=destination_agent_id,
        flow_direction=flow_direction,
        priority=priority,
        required_skills=required_skills,
        input_data=input_data
    )

    return task

def execute_task(self, task: Task) -> A2AMessage:
    """Execute a task by sending A2A message"""

    sender_card = self.agent_cards.get(task.sender_agent_id)
    if not sender_card:
        raise ValueError(f"Sender agent {task.sender_agent_id} not found")

    # Update task status
    self.task_manager.update_task_status(task.task_id, TaskStatus.IN_PROGRESS)

```

```

# Create and return A2A message
return self.a2a_protocol.create_task_message(task, sender_card)

def get_agent_capabilities(self, agent_id: str) -> Dict:
    """Get comprehensive capabilities of an agent including inherited ones"""

    agent_card = self.agent_cards.get(agent_id)
    if not agent_card:
        return {}

    inherited_capabilities = self.agent_hierarchy.get_capability_inheritance(agent_id)

    return {
        "agent_id": agent_id,
        "direct_capabilities": agent_card.capabilities,
        "inherited_capabilities": inherited_capabilities,
        "skills": {
            skill_id: {
                "name": skill.name,
                "description": skill.description,
                "keywords": skill.keywords
            } for skill_id, skill in agent_card.skills.items()
        },
        "agent_card": agent_card.to_json()
    }

# Global A2A system
a2a_system = A2ASystem()

```